

Controle de Concorrência

Tipos de Controle de Concorrência

Existem dois principais tipos de controle de concorrência:

Controle Otimista:

- Assume que conflitos são raros.
- Verifica conflitos apenas no momento de confirmar a transação.
- Ideal para sistemas com baixa concorrência.

Controle Pessimista:

- Assume que conflitos são comuns.
- Usa bloqueios (locks) para evitar acessos simultâneos.
- Ideal para sistemas com alta concorrência.

Implementação de Controle Pessimista

O controle pessimista utiliza bloqueios para evitar que múltiplas transações acessem os mesmos dados ao mesmo tempo.

1. Use Transações no Banco de Dados:

Certifique-se de que o banco de dados suporta transações (ex.: MySQL, PostgreSQL).

Inicie uma transação antes de realizar operações críticas

```
BEGIN TRANSACTION;
```

2. Aplique Locks (Bloqueios):

Use bloqueios para garantir que apenas uma transação acesse os dados:

```
SELECT * FROM tabela WHERE id = 1 FOR UPDATE;
```

O comando `FOR UPDATE` bloqueia a linha até que a transação seja concluída.

3. Confirme ou Reverta a Transação:

Após concluir as operações, finalize a transação:

```
COMMIT;
```

Em caso de erro, reverta:

```
ROLLBACK;
```

Implementação de Controle Otimista

O controle otimista verifica conflitos apenas no momento de salvar as alterações.

1. Adicione uma Coluna de Versão:

Adicione uma coluna `versao` na tabela para rastrear alterações:

```
ALTER TABLE tabela ADD COLUMN versao INT DEFAULT 0;
```

2. Leia os Dados com a Versão:

Leia os dados e armazene o valor da coluna `versao`.

3. Atualize Somente se a Versão Não Mudou:

Antes de salvar, verifique se a versão ainda é a mesma:

```
UPDATE tabela  
SET coluna = 'novo_valor', versao = versao + 1  
WHERE id = 1 AND versao = 0;
```

Se a versão mudou, significa que outra transação alterou os dados, e você pode tratar o conflito.

Atividade Prática 3

Vamos incluir o controle de concorrência “otimista” no sistema de solicitação de compras que estamos implementando.

Adicionar uma Coluna para o controle de concorrência

Adicione uma coluna `versao` na tabela `SOLICITACAO_COMPRAS`:

```
ALTER TABLE SOLICITACAO_COMPRAS ADD COLUMN VERSAO INT DEFAULT 0;
```

Leia (select) os dados e armazene a versão:

Ao buscar uma solicitação, armazene o valor da coluna `versao`.

Realize a atualização (update) somente se a Versão Não Mudou:

Antes de salvar a atualização, verifique se a versão ainda é a mesma:

```
UPDATE SOLICITACAO_COMPRAS  
SET ID_USUARIO = ?, VERSAO = VERSAO + 1  
WHERE ID = ? AND VERSAO = ?;
```

Se a versão mudou, significa que outra transação alterou os dados, e você pode tratar o conflito.

Implementando o controle de concorrência no exemplo da Solicitação de Compras

Ao atualizar uma solicitação de compra:

```
async atualizarSolicitacao(idSolicitacao, idUsuario, versao) {
  const query = `
    UPDATE SOLICITACAO_COMPRAS
    SET ID_USUARIO = ?, VERSAO = VERSAO + 1
    WHERE ID = ? AND VERSAO = ?;
  `;
  const [result] = await this.conexao.query(query, [idUsuario, idSolicitacao, versao]);

  if (result.affectedRows === 0) {
    throw new Error('Conflito detectado: os dados foram alterados por outra transação.');
```

Alterando o endpoint no OrquestradorSolicitacao:

Adapte a lógica para capturar e verificar a versão.

```
static async atualizarSolicitacao(idSolicitacao, idUsuario, versao) {
  const transacao = new TransacaoBanco();
  try {
    await transacao.iniciarTransacao();
    const conexao = transacao.connection;

    const solicitarCompraService = new
    SolicitarCompraService(conexao);
    await solicitarCompraService.atualizarSolicitacao(idSolicitacao,
    idUsuario, versao);

    await transacao.commit();
    console.log('Solicitação atualizada com sucesso!');
  } catch (erro) {
    await transacao.rollback();
    throw new Error('Erro ao atualizar a solicitação: ' +
    erro.message);
  }
}
```

Ajustando as Rotas

No arquivo de rotas altere o trecho a seguir, adicionando o tratamento para o novo campo “versão”:

```
...

// Atualizar uma solicitação de compra
router.put('/update/:id', async (req, res) => {
  const idSolicitacao = req.params.id;
  const { id_usuario } = req.body;
```

```
    const { versao } = req.body;

    try {
      await OrquestradorSolicitacao.atualizarSolicitacao(idSolicitacao,
id_usuario, versao);
      res.status(200).send('Solicitação atualizada com sucesso!');
    } catch (erro) {
      res.status(500).send('Erro ao atualizar solicitação: ' + erro.message);
    }
  });
};

...

```

Testar o Controle de Concorrência

Simule acessos simultâneos ao sistema usando ferramentas como **Postman**.

Verifique se a verificação da versão estão funcionando corretamente.

Exemplo de requisição para atualização da solicitação de compras: